`rocq-piler`: A Content-Addressed Proof Oracle for LLM-Driven Discharge of Separation-Logic Obligations

Gavin Mendel-Gleason
Scidonia, Dublin, Ireland
gavin@scidonia.com

June 19, 2026

Abstract

We present `rocq-piler`, a tool that connects Large Language Models (LLMs) to the Rocq proof assistant (formerly Coq) through the Model Context Protocol (MCP), and that is designed around a single organising idea: *content-addressed proof obligations*. Every open goal in a proof is identified by a stable hash of its statement and hypotheses, so an LLM can target, batch, and re-target obligations by hash rather than by fragile line numbers or bullet positions. On top of this substrate `rocq-piler` provides higher-level moves—`stratify` (case-split a proof and discharge the easy cases with a tactic portfolio) and `close_admits` (batch-close the survivors)—together with speculative execution and an incremental verification cache. Our motivating application is the deductive verification system `axiomander`, which compiles imperative programs to Iris separation-logic proof obligations. Automation discharges most of these obligations, but a residual fraction (in our experience roughly one in five) requires an expert. `rocq-piler` is the *oracle* we are tuning to play that expert role. As a self-contained, reproducible case study we report an autonomous proof of type preservation for PCF with mutable references (21 inductive cases, 9 supporting lemmas), completed by two different LLMs through `rocq-piler`'s MCP interface at a cost of a few cents per run. The tool is open-source and under active development.

Keywords: Proof assistants, Rocq, Coq, separation logic, Iris, LLMs, deductive verification, proof automation, MCP, neurosymbolic AI

1 Introduction

Deductive program verifiers such as those built on Iris [2] reduce the correctness of imperative programs to a set of *proof obligations* in separation logic. Powerful automation can dispatch many of these obligations, but not all: a residual tail of obligations—arising from non-trivial framing, ghost state, invariants, or arithmetic side conditions—is left for a human expert. This tail is precisely where verification projects stall, because it demands scarce expertise and tedious interactive proof.

`rocq-piler` targets this tail. It is a tool that exposes the Rocq proof assistant [1] to Large Language Models (LLMs) through the Model Context Protocol (MCP), with the explicit goal of acting as an *oracle* that an automated verifier can call when its own tactics give up. The LLM proposes proof steps based on learned patterns; Rocq, accessed through its Language Server Protocol interface [5] and the Pétanque execution backend, verifies every step. This division of labour—neural proposal, symbolic verification—means the LLM need never be trusted: unsound steps are simply rejected.

The novel element of `rocq-piler` is not the bridge itself but its *interface abstraction*. Earlier LLM–prover integrations address goals positionally (line/column, bullet nesting), which is brittle: a single inserted tactic invalidates every downstream coordinate, and multi-goal proofs

quickly become unmanageable for an agent. `rocq-piler` instead makes every open obligation *content-addressed*: each goal is identified by a stable hash derived from its statement and hypotheses. Identical goals share a hash and can be closed together; an obligation can be revisited later by the same hash even after the surrounding script has changed. This turns proof construction into a simple loop—*enumerate hashes, write a tactic for each hash, repeat until* `Qed`—that is robust under edits and natural for an autonomous agent.

Contributions.

- A **content-addressed obligation model** for interactive proof, in which open goals are referenced by stable hashes rather than positions (Section 3).
- Two **batch proof moves** built on this model—`stratify` (skeleton case-split plus a per-subgoal closing portfolio) and `close_admits` (portfolio-based batch closing of survivors)—plus speculative execution and an **incremental verification cache** that re-checks only changed functions and their affected callers (Section 4).
- An account of `rocq-piler` as a **proof oracle for** `axiomander`, an open deductive verifier that emits Iris separation-logic obligations, where roughly one fifth of obligations resist automation and fall to the oracle (Section 2).
- A **reproducible case study**: autonomous proof of type preservation for PCF with references (21 cases, 9 lemmas) by two distinct LLMs, with cost and tool-call statistics (Section 5).

Availability: `rocq-piler` is open-source (MIT license), available at `https://github.com/scidonia/rocq-piler` and published on npm as `rocq-piler`.

2 Motivation: the Residual Obligations of a Deductive Verifier

`axiomander` is a separate open-source deductive verification system. It takes annotated imperative code and produces proof obligations in Iris, a higher-order concurrent separation logic embedded in Rocq [2]. Like other verifiers in this family, `axiomander` discharges a large majority of its obligations automatically through built-in tactics and heuristics. The difficulty is the remainder.

In our experience, automation alone leaves on the order of **20% of obligations** unproved. These are the obligations that require genuine proof engineering: choosing how to frame a heap assertion, supplying an existential witness for ghost state, strengthening a loop invariant, or closing an arithmetic side goal that the automation cannot see through. Historically this 20% is handled by a human expert working interactively in Rocq—the bottleneck that limits how much code a verification effort can cover.

`rocq-piler` is our attempt to automate that expert. Rather than extending `axiomander`'s internal automation, we treat the leftover obligations as ordinary Rocq goals and hand them to an LLM-driven oracle that interacts with Rocq through `rocq-piler`. The verifier remains responsible for soundness—every obligation is still a Rocq goal checked by Rocq's kernel—while the oracle supplies the creative, hard-to-automate proof steps. Because the oracle is an LLM behind an MCP interface, it can be *tuned*: the tool surface, the prompting, the closing portfolios, and the model itself are all parameters we adjust to raise the fraction of the residual obligations that close without a human.

This integration is an early-stage prototype. In this paper we therefore use a self-contained Rocq development—type preservation for PCF with references—as a faithful, reproducible stand-in for the kind of obligation the oracle must close: deep induction, auxiliary lemmas about substitution and weakening, and reasoning about a mutable store. It exercises exactly the

oracle behaviours that the `axiomander` tail requires, without depending on the full `axiomander` pipeline.

3 The Content-Addressed Obligation Model

3.1 From positions to hashes

A Rocq proof in progress is a tree of open goals beneath a partially written script. Addressing those goals by position is fragile: inserting a tactic shifts every subsequent line, and Rocq's bullet discipline ($-$, $+$, $*$, braces) makes the "current" goal depend on subtle structural context. For an autonomous agent that edits the script repeatedly, positional addressing is a constant source of stale references.

`rocq-piler` replaces positions with content. Each open obligation is sealed as a named `admit` annotated with an 8-character hash computed from the goal's statement and its hypothesis context. The hash has two properties that drive the whole workflow:

- **Stability**: the hash depends only on the goal, not on where it sits in the script. An obligation can be revisited by the same hash after unrelated edits elsewhere.
- **Coalescence**: syntactically identical goals share a hash. A proof that produces four copies of the goal `True`, possibly at different bullet depths, exposes a single hash that closes all four at once.

3.2 The core loop

Proof construction with `rocq-piler` reduces to a short cycle:

1. `focus_proof` returns the proof state, the script so far, and the list of open obligations—each with its hash, hypotheses, and goal.
2. For each hash, `insert_tactics` commits a tactic (or a multi-line block) targeting that obligation by hash. If the tactic leaves several goals, they are re-sealed as fresh hash-addressable admits.
3. Repeat until no obligations remain, at which point the proof is closed automatically with `Qed`.

Crucially, the agent reads hypotheses-plus-goal for every obligation directly from the `focus_proof` (or `insert_tactics`) response, so it can write each case's tactic without re-querying the prover for context.

4 Tool Architecture and Higher-Level Moves

4.1 System overview

`rocq-piler` is implemented in TypeScript on top of three layers:

1. an **MCP server** exposing the tool surface to any MCP-compatible client (e.g. Claude, OpenCode, custom agents);
2. an **LSP/Pétanque client** that drives `rocq-lsp` for document synchronisation, diagnostics, and speculative state execution; and
3. a **workspace manager** that detects project roots (`_CoqProject`, `_RocqProject`, `dune-project`) and keeps the language server aligned with the right project.

4.2 Core obligation tools

- `focus_proof`: one-stop entry point—returns goal state, the script up to the cursor, and every open admit with its hash, hypotheses, and goal.
- `insert_tactics`: insert a tactic or multi-line block; target a specific obligation with `admit_hash`; supports speculative `dry_run` evaluation and `replace` for retry.
- `open_goals` / `proof_state`: lighter-weight goal queries.
- `add_lemma` / `delete_lemma`: insert or remove helper lemma stubs above a target proof.
- `reset_proof` / `undo_step` / `delete_step`: structured rollback of a proof body or of recent edits.

4.3 Batch moves: `stratify` and `close_admits`

Two moves make multi-case proofs tractable for an agent.

`stratify` runs a *skeleton* tactic that case-splits the proof (for example, `induction H; intros; inversion Ht; subst`), then attempts each resulting subgoal independently with every tactic in a *closing portfolio*, each wrapped in `solve[...]` so that partial progress never leaks between cases. Solved cases have their winning tactic inlined; unsolved cases become labelled, hash-addressable admit bullets. A single `stratify` call can thus dispatch the routine majority of an induction and report only the genuine survivors, by hash.

`close_admits` takes a portfolio mapping hashes (or the wildcard `*`) to closing tactics and applies them speculatively: a tactic is committed only if it closes its target obligation, otherwise the obligation is left untouched. This lets the agent sweep up survivors in one call while guaranteeing the script never regresses.

Together these moves implement the natural strategy for a hard goal: split it, auto-close what you can, and spend the LLM's attention only on the residue—the same shape as the `axiomander` problem of automating the 20% tail.

4.4 Speculative execution and knowledge search

`rocq-piler` exposes the Pétanque API (`snap_state`, `exec_tactic`, `state_goals`) and a `dry_run` mode so an agent can test a tactic without modifying the file. Knowledge tools—`search_lemmas`, `inspect_term`, `inspect_about`, `locate_term`, `require_lib`—let the agent discover applicable lemmas and definitions before committing.

4.5 Incremental verification cache

Because an oracle is invoked repeatedly over an evolving development, `rocq-piler` caches verification at the granularity of individual functions/proofs. It tracks separate hashes for a proof's body, its contract/statement, and the contracts of its callees, and re-verifies only what changed plus the transitive callers affected by a contract change. Unchanged, previously verified obligations return instantly. This keeps the cost of repeatedly consulting the oracle low—an essential property when it is wired into a verifier's inner loop.

Table 1: Selected case-study statistics (DeepSeek V4 run).

Inductive step cases	21
Auxiliary lemmas	9
Cases auto-closed by a single `stratify`	18
Hash-addressed survivors closed individually	3
Total API calls	50
Actual provider cost	$0.04

5 Case Study: Type Preservation for PCF with References

5.1 Problem

As a reproducible proxy for the `axiomander` residual obligations, we verify type preservation for PCF extended with mutable references:

$$\frac{\Gamma; \Sigma \vdash e : \tau \quad (e, \mu) \to (e', \mu') \quad \mu : \Sigma}{\exists\, \Sigma' \supseteq \Sigma.\ \ \mu' : \Sigma' \ \wedge\ \Gamma; \Sigma' \vdash e' : \tau}$$

The development includes a typed store, store-typing extension, and a de Bruijn substitution calculus. The proof requires 21 inductive step cases and 9 supporting lemmas covering store extension (`extends_refl`, `extends_trans`, `nth_error_extends`), weakening (`store_weakening`, `heap_ok_extends`), shifting and substitution (`shift_typing`, `subst_preserves_typing`), and heap invariants (`heap_lookup_has_type`, `heap_update_ok`).

5.2 Autonomous process

The proof was driven entirely through `rocq-piler`'s MCP interface. The agent proceeded as the architecture intends:

1. Introduce auxiliary lemma stubs with `add_lemma` and prove each by the focus/insert loop; the substitution and shifting lemmas required a generalised induction (over an arbitrary context prefix) that the agent discovered after an initial formulation failed and was reset.

2. Attack preservation with a single `stratify`: a skeleton induction over the step relation plus a closing portfolio, which discharged 18 of the 21 cases automatically and left three survivors (`S_AppAbs`, `S_DerefLoc`, `S_AssignV`) addressed by hash.

3. Close the three survivors individually with `insert_tactics` targeted by `admit_hash`, invoking the substitution and heap lemmas.

5.3 Results

The full development closes with `Qed` on all 9 lemmas and the main theorem, with no remaining admits. We ran the task with two distinct models—an open-weight model (DeepSeek V4) and a proprietary model (a Claude release)—each completing the proof through the same MCP interface. A representative DeepSeek V4 run used **50 API calls** at an actual provider cost of **about four cents** ($0.04). The `stratify`/`close_admits` pattern was decisive: the bulk of the 21 cases never reached the model as individual problems, mirroring the goal of letting the oracle spend effort only on the genuinely hard obligations.

6 Related Work

Separation-logic verification. Iris [2] underpins a family of deductive verifiers whose automation leaves a residual tail of interactive obligations; `axiomander` sits in this family, and `rocq-piler` targets that tail rather than the logic itself.

LLM-based theorem proving. Baldur [3] generates and repairs Coq proofs with retrieval-augmented LLMs, and LeanDojo [4] provides a learning environment for Lean. These focus on model training and bespoke environments; `rocq-piler` instead standardises the *interaction protocol* (MCP) and contributes the content-addressed obligation model and batch moves, so any MCP-capable model can serve as the oracle.

Prover interfaces. `coq-lsp` [5] provides the LSP foundation `rocq-piler` builds on; earlier IDE-oriented tools such as CoqPIE [6] and Proof General [7] target human users. `rocq-piler` adds an agent-oriented layer: hash-addressed goals, speculative execution, batch closing, and an incremental cache.

7 Status, Lessons, and Future Work

The `axiomander` integration is a working prototype: leftover Iris obligations can be routed to the oracle, and we are actively tuning the tool surface, prompting, and closing portfolios to increase the share of the 20% tail that closes without a human. Two design lessons stand out. First, *addressing obligations by content rather than position* is what makes autonomous, multi-goal proof construction robust to the agent's own edits. Second, *batch moves with speculative, non-regressing semantics* (`stratify`, `close_admits`) match how the residual-obligation problem actually decomposes: auto-close the routine majority, escalate only the survivors.

Future work includes (i) end-to-end evaluation on real `axiomander` obligation suites with success-rate and cost reporting; (ii) feeding verifier-side context (framing hints, ghost-state signatures) into the oracle's prompts; and (iii) learning closing portfolios from past successful discharges.

8 Conclusion

`rocq-piler` reframes LLM-driven proving around content-addressed obligations and batch, speculative closing moves, and positions the result as a tunable oracle for the hard tail of a separation-logic verifier, `axiomander`. A reproducible PCF+references case study—21 cases and 9 lemmas, closed autonomously by two different models for a few cents—demonstrates the workflow on exactly the kind of goal the oracle must handle. The tool is open-source and under active development.

Tool availability: `https://github.com/scidonia/rocq-piler`

References

[1] The Coq Development Team: The Coq Proof Assistant. `https://doi.org/10.5281/zenodo.14542673` (2024)

[2] Jung, R., Krebbers, R., Jourdan, J.-H., Bizjak, A., Birkedal, L., Dreyer, D.: Iris from the ground up: A modular foundation for higher-order concurrent separation logic. Journal of Functional Programming **28**, e20 (2018)

[3] First, E., Rabe, M.N., Ringer, T., Brun, Y.: Baldur: Whole-Proof Generation and Repair with Large Language Models. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) (2023)

[4] Yang, K., Swope, A.M., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R., Anandkumar, A.: LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)

[5] Gallego Arias, E.J., et al.: Rocq-lsp: Visual Studio Code Extension and Language Server Protocol for Rocq, v0.2.5. `https://github.com/ejgallego/rocq-lsp` (2025)

[6] Roe, K.: CoqPIE: A Plugin for Integrated Development Environments. In: Conference on Intelligent Computer Mathematics (CICM) (2016)

[7] Aspinall, D.: Proof General: A Generic Tool for Proof Development. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (2000)